PROJECT ON:

RecycleNet: Deep Learning Assisted Dynamic Plastic Identification Technique For Automated Waste Segregation

Final Report Submitted in Partial Fulfilment of the Requirements of the Degree of Bachelor of Technology

In

Electronics and Instrumentation Engineering

By

Sparsh Kumar Gupta (Roll no:13005322030)

Under The Guidance Of

Saikat Majumdar
Faculty of EIE Dept
Techno India, Saltlake



Techno India,Salt Lake
EM-4/1, Sector-V, Salt Lake City, Kolkata:700091 2018

MAULANA ABUL KALAM AZAD UNIVERSITY OF TECHNOLOGY (MAKAUT)

TECHNO MAIN SALT LAKE

MAULANA ABUL KALAM AZAD UNIVERSITY OF TECHNOLOGY (MAKAUT)

Faculty of EIE Department

Certificate Of Recommendation

This is to certify that Sparsh Kumar Gupta has been involved in this project work titled "**RecycleNet: Deep Learning Assisted Dynamic Plastic Identification Technique For Automated Waste Segregation** ", under the direct supervision and guidance of Saikat Majmuder. I am satisified with his work, which is being presented for the partial fullfilment of the degree of Bachelor of Technology in Electronics And Instrumentation Engineering, Maulana Abul Kalam Azad University Of Technology, West Bengal.

.....

Dr. Saikat Majumder

Head Of Department

Department Of Electronics & Instrumentation Engineering

.....

Dr. Saikat Majumder

(Project Mentor)

MAULANA ABUL KALAM AZAD UNIVERSITY OF TECHNOLOGY (MAKAUT)

Faculty of EIE Department

Provisional Certificate of Approval *

The foregoing project synopsis is hereby approved as a creditable study of Bachelor of Technology and presented in a mannersatisfactory to warrantits acceptance as a pre-requisite to the final semester examination for which it has been submitted. It is understood that by this approval the undersigned do not necessarily endorse or any statement made, opinion expressed or conclusion therein but approve this project only for the purpose for which it is submitted

.....

Signature of the examiners

ABSTRACT

Effective solid waste management is an increasingly vital field that merges advanced computer vision, edge intelligence, and automated control frameworks to optimize recycling efficiency. Traditional physical separation networks depend heavily on human visual labor, which introduces high latency and health errors, or sophisticated near-infrared (NIR) and spectrometer arrays that suffer from prohibitive implementation costs.

This project presents RecycleNet, a scalable, high-speed, and accessible deep learning architecture engineered to automate real-time industrial resin type detection. Utilizing a custom physical dataset containing 2,420 unique high-resolution images distributed across 8,884 bounding box labels, the network models are tightly trained around the industry-standard six Resin Identification Codes: PET, HDPE, PVC, LDPE, PP, and PS.

We benchmark edge-focused deployment paradigms utilizing the latest state-of-the-art YOLO26 nano (YOLO26n), YOLOv8 small (YOLOv8s), YOLO26 medium (YOLO26m), and RF-DETR transformers. While the transformer-based RF-DETR engine delivers peak structural accuracy with a mean Average Precision (mAP@50) of 62.0% at an inference loop of 14.8 ms, the edge-optimized YOLO26n model outputs a blistering 2.0 ms processing turnaround per structural tensor grid with 58.6% mAP@50.

To bridge computer vision algorithms into functional engineering environments, a robust multi-platform network ecosystem was built. The ecosystem features a cross-platform Python Flask server interface paired with native Android CameraX and PyTorch Lite implementations for true real-time standalone mobile operational sorting.

INDEX

Chapter 1: Introduction 5

The Plastic Segregation Challenge

Limitations of Legacy Industrial Methods

Deep Learning Component Frameworks (YOLO26 & DETR)

The RecycleNet Computer Vision Approach

Objective of the Project

Advantages of the System

Chapter 2: Project Block Diagram 8

System Architecture and Workflow Overview

Chapter 3: Detailing of the Project 10

Custom Visual Sampling Matrix

Roboflow Class Bounding & Annotation

Data Augmentation Strategies

Computational Training Infrastructure

Resin Class Matrix Breakdown

Cross-Platform Desktop Flask Pipeline

OpenCV Loop

Native Kotlin and Jetpack Compose Mobile Frontend

Android CameraX Live Stream Loop Analyzer

PyTorch Lite Runtime Conversion (.PTL Model)

Chapter 4: Results and Discussions 15

Model Metrics

Confusion Matrix

Insights from Analytics

Chapter 5: Conclusion & Future Scope 22

Project Conclusion

Future Scope

Chapter 6: Appendix 24

Getting Started with Installation (Android)

Getting Started with Installation (Desktop)

Chapter 7: References 27

1.INTRODUCTION

1.1 THE PLASTIC SEGREGATION CHALLENGE

Plastic waste accumulation forms one of the most critical contemporary ecological crises. Millions of tons of post-consumer polymers are routed directly into maritime ecosystems and sanitary landfills every year. Sorting this complex municipal output waste stream at the recovery facility remains the core logistical obstacle preventing high-yield polymer loop recovery.

1.2 LIMITATIONS OF LEGACY INDUSTRIAL METHODS

Existing sorting systems display a direct, limiting compromise between economic feasibility and throughput precision:

- **Manual Separation Sorting:** Highly inefficient, subjective, error-prone, and exposes workers to biohazards and chemical trace toxins.
- **Near-Infrared (NIR) Spectrometry:** Offers high specificity across diverse chemical matrices, but its immense unit costs restrict deployment to highly capitalized centralized sorting facilities.

1.3 DEEP LEARNING COMPONENT FRAMEWORKS (YOLO26 & DETR)

Recent paradigm shifts in computer vision have made real-time structural identification on cheap edge devices feasible. This project implements the recently engineered YOLO26 framework alongside RF-DETR transformers. YOLO26 introduces structural architecture modifications that optimize multi-scale feature aggregation layers by eliminating traditional Non-Maximum Suppression (NMS) latency loops. This allows direct end-to-end multi-object regression mapping, enabling ultra-low inference times on low-power mobile microprocessors.

1.4 THE RECYCLENET COMPUTER VISION APPROACH

RecycleNet combines modern deep learning architectures with cross-platform edge execution frameworks to create an automated, real-time visual sorting solution. By processing real-time video streams, it identifies, creates a bounding box around, and classifies individual plastic targets based on their polymer signatures.

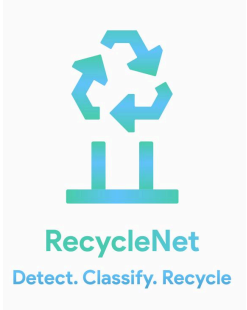


Figure 1.1 RecycleNet LOGO

1.5 OBJECTIVE OF THIS PROJECT

The central target goals of this instrumentation project are:

- To curate, organize, and compile a completely custom, diverse, and clean target dataset containing 2,420 multi-angle post-consumer plastic waste samples.
- To evaluate structural precision parameters across YOLO26n, YOLOv8s, YOLO26m, and RF-DETR deep learning configurations.
- To assemble a unified, low-latency multi-threaded Python Flask backend server for local system sorting integration.
- To build a native Android execution wrapper using Google Jetpack Compose, Kotlin, CameraX API, and PyTorch Lite runtimes (.ptl format) for entirely offline mobile deployment.
- To be able to detect materials like HDPE, LDPE,PVC, PP, PS, PET



Figure 1.2- 6 Classes Of Plastic Material

1.6 ADVANTAGES OF THE SYSTEM

- **Extremely Low Unit Capital:** Operates seamlessly via consumer-grade webcams and mobile camera modules, replacing expensive sensors.
- **True Edge Autonomy:** The mobile application variant works with zero cloud network connectivity requirements, enabling distributed, grid-free operation.
- **High Real-Time Throughput:** Latency metrics down to 2.0 ms enable rapid physical sorting conveyor systems.

2.PROJECT BLOCK DIAGRAM

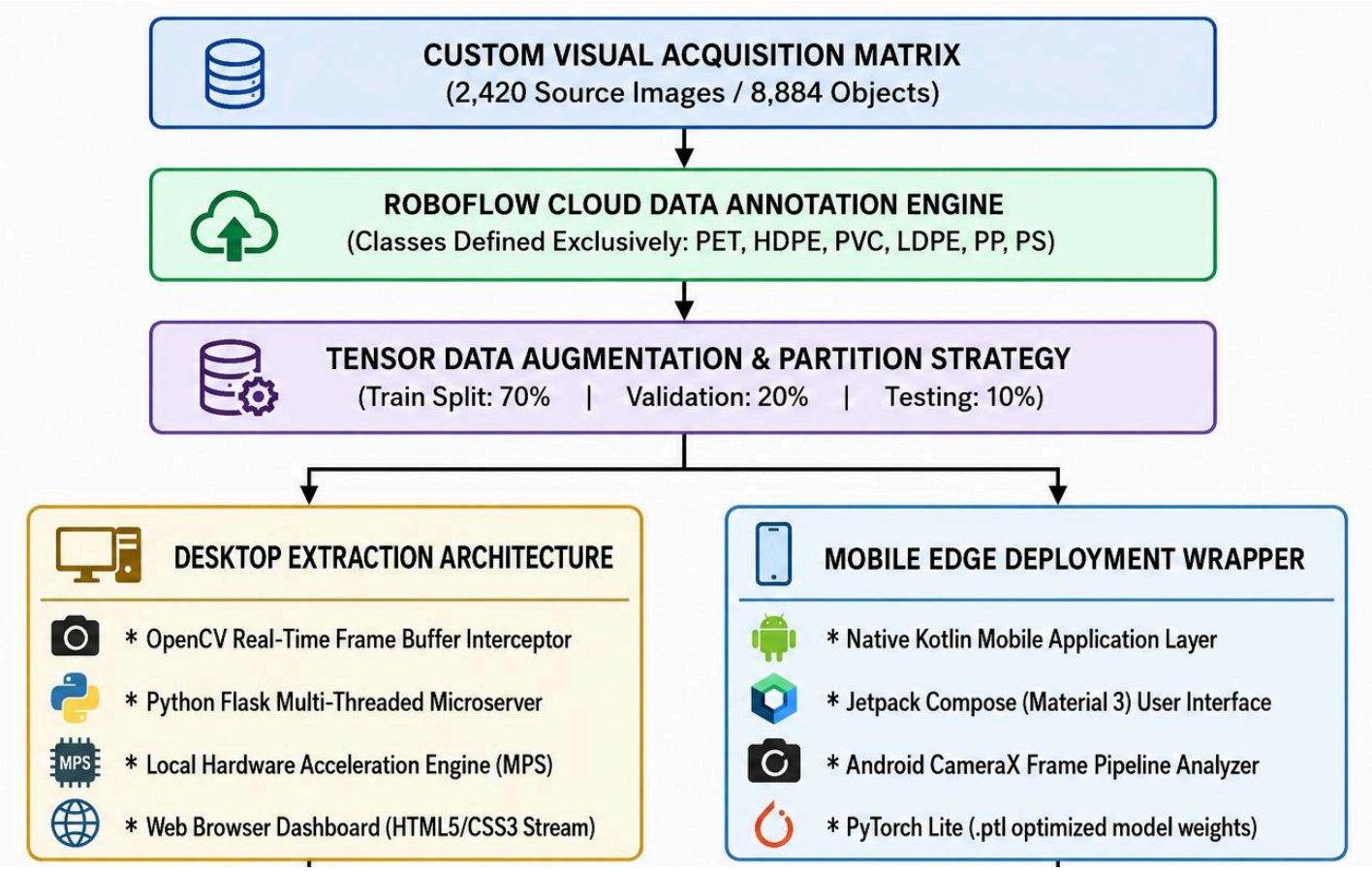


Figure 1.3: Unified Functional Layout Block Diagram of the RecycleNet Detection & Execution Pipeline.

2.1 System Architecture and Workflow Overview

Figure 1.3 illustrates the overall architecture and operational workflow of the proposed intelligent plastic waste classification and segregation system. The process begins with the acquisition of a custom image dataset containing 2,420 source images and 8,884 annotated objects. The dataset is labeled using the Roboflow cloud annotation platform across six plastic categories: PET, HDPE, PVC, LDPE, PP, and PS. Following annotation, the data undergoes augmentation and partitioning into training, validation, and testing subsets to facilitate robust model development and evaluation. The trained model is subsequently deployed through two independent execution environments: a desktop-based architecture featuring OpenCV, Flask, hardware acceleration, and a web dashboard, and a mobile edge deployment framework utilizing Kotlin, Jetpack Compose, CameraX, and PyTorch Lite. Both deployment pipelines perform real-time object detection and classification, ultimately supporting an automated segregation mechanism that identifies plastic types and enables category-specific sorting actions. This workflow provides a scalable and efficient solution for intelligent waste management through computer vision and edge AI technologies.

3.DETAILING OF THE PROJECT

3.1 CUSTOM VISUAL SAMPLING MATRIX

Because public datasets lack dense class representations for all industrial resin categories, a completely custom dataset was generated from scratch. It contains 2,420 unique images of diverse multi-use containers under varying deformation conditions and illumination angles.

3.2 ROBOFLOW CLASS BOUNDING & ANNOTATION

Image labeling was managed inside the Roboflow cloud ecosystem. A total of 8,884 high-precision bounding boxes were annotated across the dataset. The label distributions maps strictly to the standard resin identification codes.



Figure 1.4: Roboflow platform used for annotations

3.3 DATA AUGMENTATION STRATEGIES

To maximize validation generalization and prevent gradient over-fitting, training batches underwent real-time geographic transformations, including horizontal/vertical tensor flips, random degrees of axial rotation, brightness scaling, and salt-and-pepper noise insertions.

3.4 COMPUTATIONAL TRAINING INFRASTRUCTURE

Model Training was performed on dedicated Kaggle and Google Colab cloud platforms utilizing high-throughput hardware architectures containing Nvidia T4 Tensor Core Graphics Processing Units (GPUs). Multi GPU Training was used to help models train faster using 2 T4 GPUs Provided by Kaggle in parallel.



Figure 1.5: Google Colab and Kaggle used for model training

3.5 RESIN CLASS MATRIX BREAKDOWN

The model targets exclusively to six plastic materials:

- PET (Polyethylene Terephthalate)
- HDPE (High-Density Polyethylene)
- PVC (Polyvinyl Chloride)
- LDPE (Low-Density Polyethylene)
- PP (Polypropylene)
- PS (Polystyrene)



Figure 1.6- 6 Classes Of Plastic Material

3.6 CROSS-PLATFORM DESKTOP FLASK PIPELINE

The local host computer variant uses a multi-threaded Python Flask micro-framework script (app.py). The script initializes system resources, structures secure local networking loops, and instantiates the chosen YOLO configuration array using system-level weight allocations (.pt files).

3.7 OPEN CV LOOP

An asynchronous processing loop is opened via OpenCV (cv2.VideoCapture). Raw incoming camera frames are resized, shifted to standard RGB color spaces, passed directly through the forward-propagation tensor layer using Apple Silicon Metal Performance Shaders (MPS) or Nvidia CUDA libraries, and streamed back to the frontend browser interface as an optimized multipart JPEG frame sequence. If both are missing, the platform defaults to utilizing CPU for inference automatically.

3.8 NATIVE KOTLIN AND JETPACK COMPOSE MOBILE FRONTEND

The smartphone edge client was written entirely in native Kotlin using Jetpack Compose with Material 3 Design Guidelines. This ensures an efficient runtime layout that consumes minimal system memory during heavy continuous processing loops.

3.9 ANDROID CAMERAX LIVE STREAM LOOP ANALYZER

The device's built-in camera array interfaces with the core application via the Android CameraX API. An automated frame analyzer module intercepts live target images inside a continuous rendering state, applying local matrix normalization routines directly before running edge network inference loops.

3.10 PYTORCH LITE RUNTIME CONVERSION (.PTL MODEL)

For offline local edge inference, standard PyTorch model matrices were compiled down into highly optimized on-device operational weights using the target PyTorch Lite (.ptl) model format. The application supports two runtime modes: Live Scan Mode (which handles continuous video classification using native Non-Maximum Suppression to filter overlapping boxes) and Gallery Mode (for processing static images).

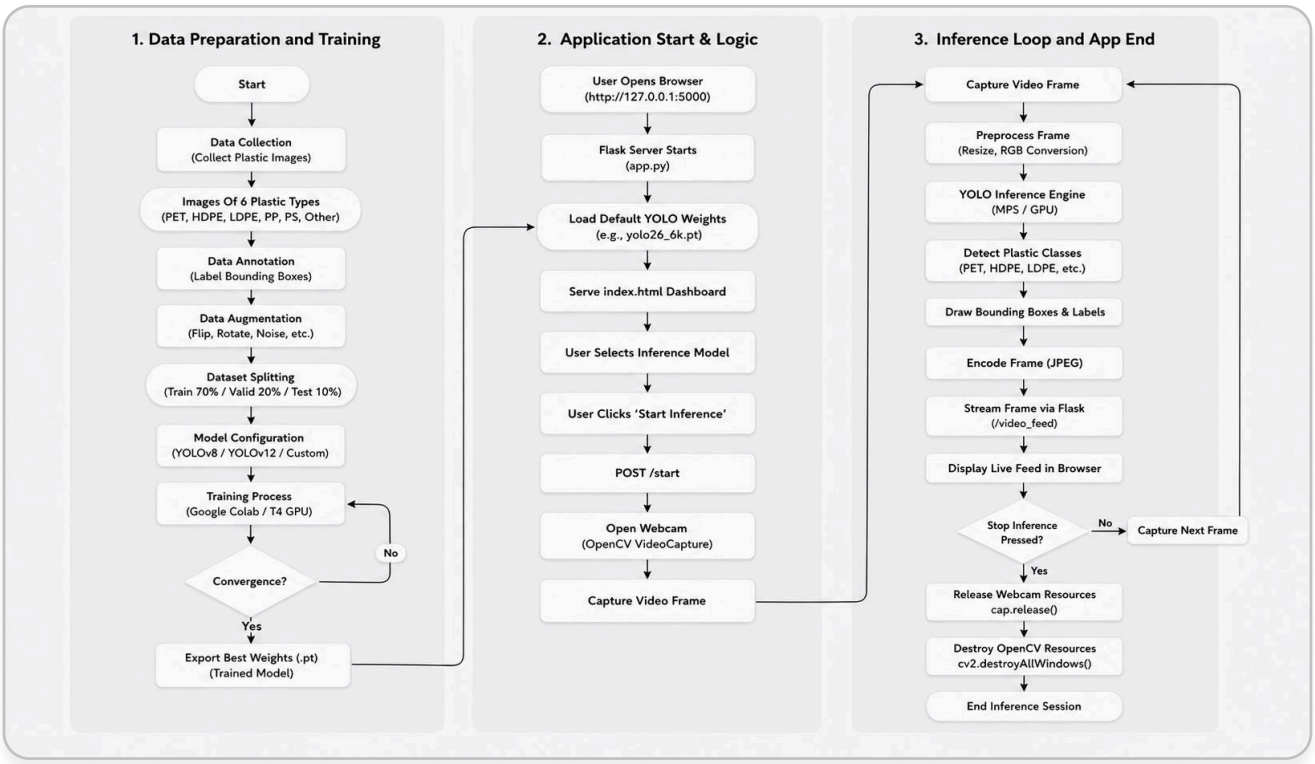


Figure 1.7- Flowchart Of RecycleNet For Desktop

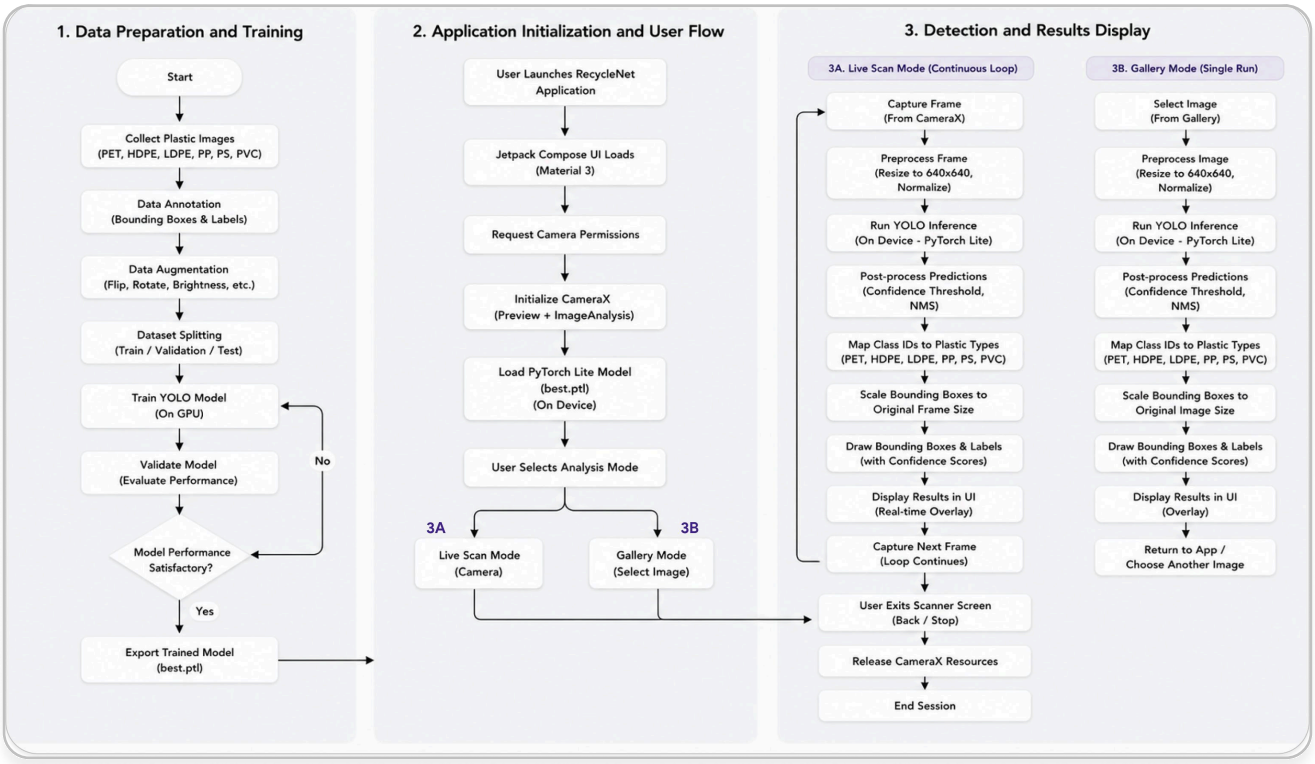


Figure 1.8- Flowchart Of RecycleNet For Mobile

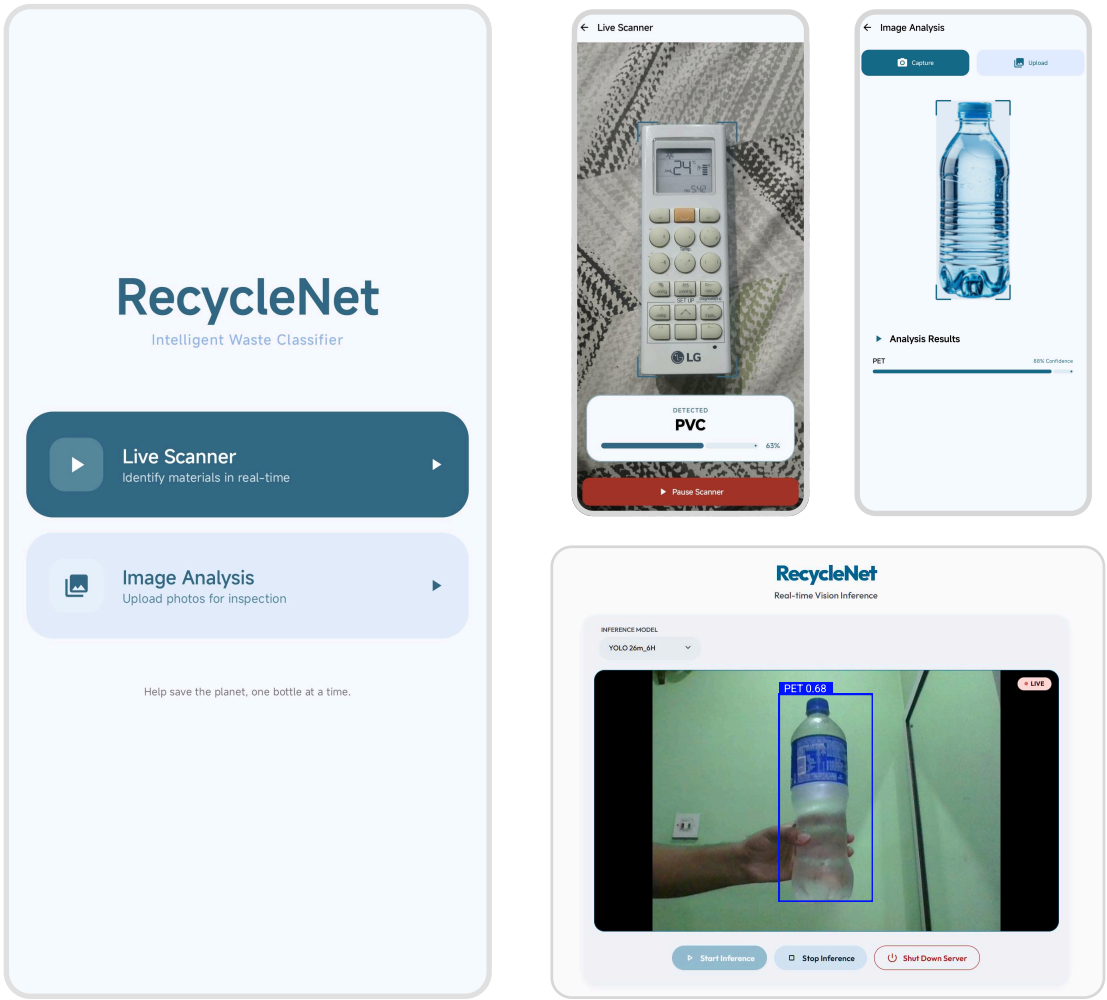


Figure 1.9- User interface of RecycleNet

4.RESULTS AND DISCUSSIONS

4.1 MODEL METRICS

The experimental results demonstrate a clear trade-off between spatial classification accuracy and computational performance across different model backbones.

Model	Params(M)	Precision	Recall	mAP50	mAP50-95	Infer(ms)
YOLO26n	2.38	0.723	0.542	0.586	0.415	2.0
YOLOv8s	11.13	0.732	0.538	0.591	0.408	2.8
YOLO26m	20.35	0.666	0.573	0.602	0.419	7.1
RF-DETR	31.99	0.74	0.591	0.62	0.457	14.8

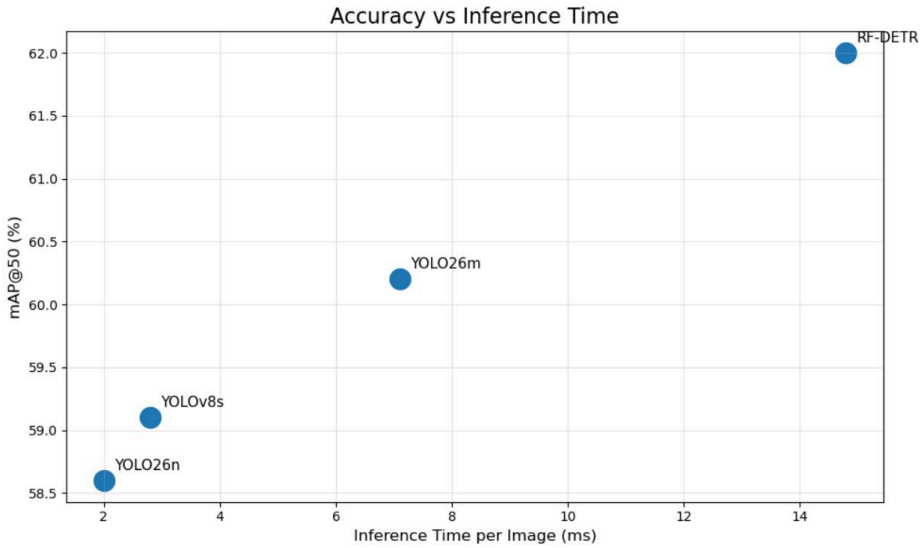
Table 1- Model Metrics

Table 1 indicates a clear trade-off between accuracy and computational efficiency.

- RF-DETR achieves the highest Precision (0.740), Recall (0.591), mAP50 (0.620), and mAP50-95 (0.457), demonstrating the best overall detection performance, but it also has the highest inference time (14.8 ms) and largest model size (31.99M parameters).
- YOLO26n is the most lightweight model (2.38M parameters) with the fastest inference (2.0 ms), making it suitable for real-time edge deployment, though with slightly lower accuracy.
- YOLOv8s provides a balanced compromise between speed and accuracy.
- Overall, RF-DETR is best for maximizing detection accuracy, while YOLO26n is best for resource-constrained, real-time applications.



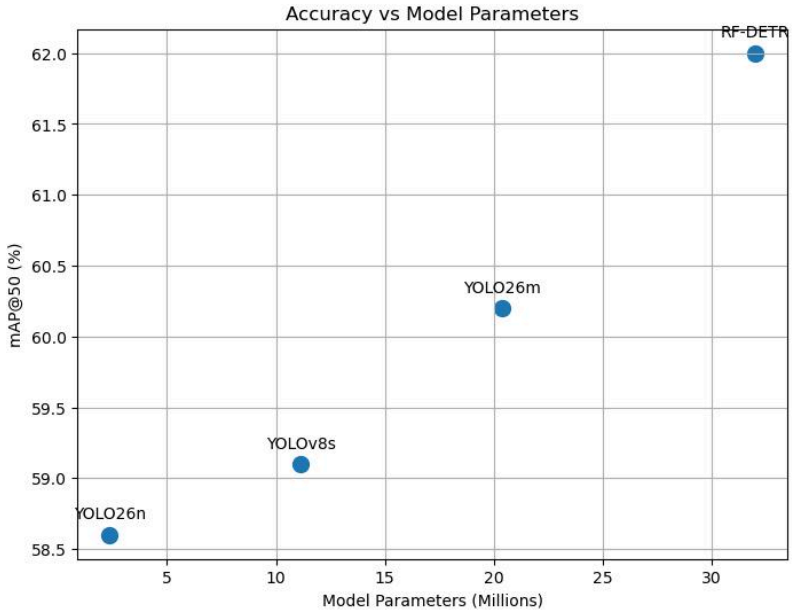
Graph 1- Per Class Heatmap



Graph 2- Accuracy vs Inference time for each model

Most Accurate Model: RF-DETR achieves the highest accuracy at 62.0% mAP@50, but it is also the slowest, requiring approximately 14.8 ms per image.

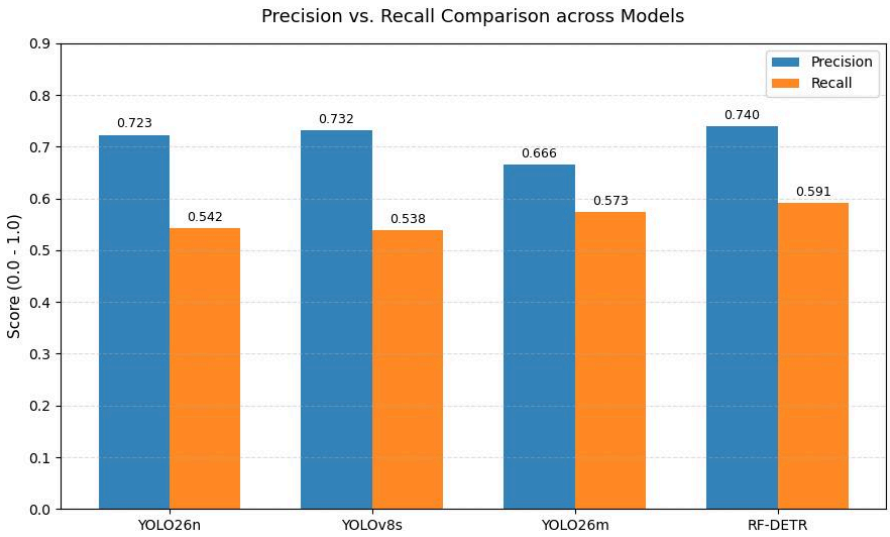
Fastest Model: YOLO26n is the fastest model with an inference time of just 2 ms, though it yields the lowest accuracy at 58.6% mAP@50.



Graph 3- Accuracy vs Model Parameter for each model

Efficiency: YOLO26n is the most "efficient" in terms of footprint, using only 2.38M parameters.

Scalability: As you increase the parameter count (moving from 2.38M to 31.99M), the accuracy gains are consistent, suggesting the architecture scales well.

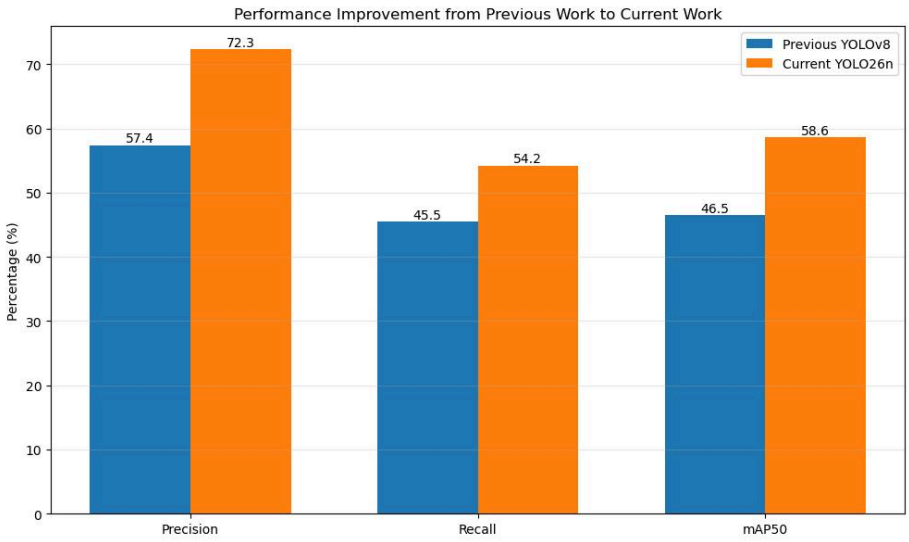


Graph 4- Precision and Recall for each model

The YOLO26m Precision Anomaly: While YOLO26m scales up in overall parameters and generic map layers, it experiences a stark drop in Precision (0.666) compared to its lighter peers. However, it gains the highest Recall (0.573) among the YOLO options. This indicates it is highly sensitive—it catches more actual objects (high recall) but throws significantly more false-positive flags (low precision).

RF-DETR holds Global Supremacy: Unlike the trade-offs observed within the YOLO architectures, RF-DETR achieves the optimal balance. It simultaneously leads the board in both Precision (0.740) and Recall (0.591), meaning it makes fewer mistakes while missing fewer true targets.

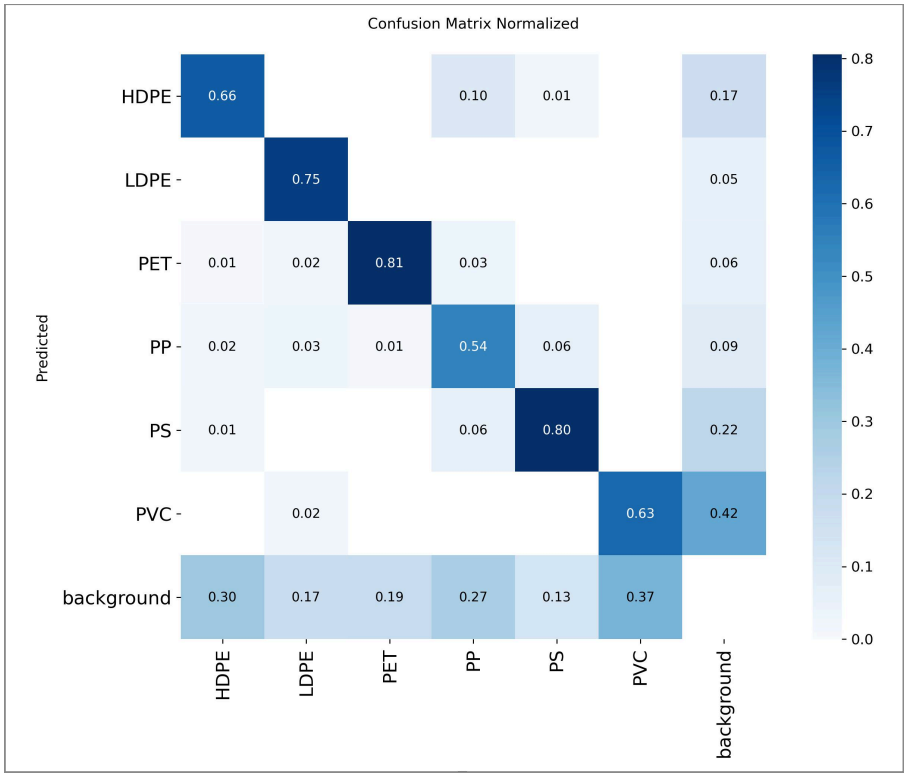
YOLO26n vs. YOLOv8s Behavior: Despite YOLOv8s being a much larger model configuration than YOLO26n, their underlying precision-recall split is nearly identical. YOLOv8s favors strictness (slight bump to 0.732 precision, small drop to 0.538 recall), showing that raw model size scaling doesn't automatically mean a balanced improvement across both sub-metrics.



Graph 5- Progress Comparison

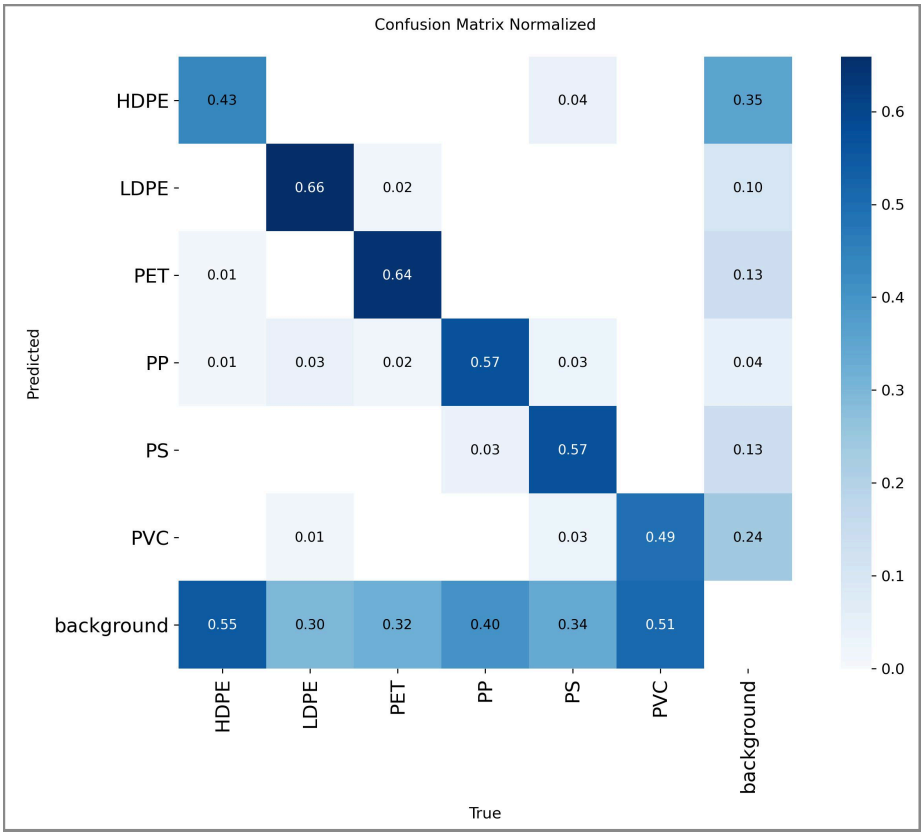
The proposed system demonstrates a substantial improvement over the previously published YOLOv8 model, increasing Precision from 57.4% to 72.3%, Recall from 45.5% to 54.2%, and mAP@50 from 46.5% to 58.6%. These gains were achieved through dataset expansion and the adoption of newer object detection architectures.

4.2 CONFUSION MATRIX



Graph 6- Confusion Matrix (RF-DETR)

Graph 6 presents the normalized confusion matrix for RF-DETR. The model demonstrates strong performance in recognizing LDPE and PET samples, as indicated by the high values along the diagonal. However, a considerable proportion of instances are classified as background, particularly for HDPE and PVC, suggesting that missed detections remain a major source of error. The matrix also reveals confusion among visually similar plastic categories, highlighting the challenges associated with distinguishing polymers based solely on visual appearance.



Graph 7- Confusion Matrix (YOLO26n)

Graph 7 presents the normalized confusion matrix for YOLO26n. The model exhibits stronger class-wise discrimination, with PET, PS, and LDPE achieving the highest recognition rates. Compared to RF-DETR, YOLO26n shows lower background confusion and more concentrated diagonal values, indicating improved consistency in object detection and classification. Although RF-DETR achieves a slightly higher overall mAP@50, YOLO26n provides a more balanced performance while maintaining significantly lower computational requirements.

4.2 INSIGHTS FROM ANALYTICS

The experimental results highlight a clear trade-off between detection accuracy and computational efficiency across the evaluated architectures. RF-DETR achieved the highest overall performance, recording an mAP@50 of 62.0% and an mAP@50-95 of 45.7%, demonstrating strong localization and classification capabilities. However, this performance comes at the cost of higher computational complexity, with the model requiring 31.99 million parameters and exhibiting the slowest inference speed of 14.8 ms per image.

YOLO26n emerged as the most balanced architecture, achieving a competitive mAP@50 of 58.6% while delivering the fastest inference speed of 2.0 ms per image using only 2.38 million parameters. The accuracy-versus-inference and parameter-versus-accuracy analyses indicate that YOLO26n maintains strong detection performance despite its significantly smaller model size, making it highly suitable for real-time deployment scenarios.

Comparison with the previously published YOLOv8 implementation further demonstrates substantial progress, with Precision improving from 57.4% to 72.3%, Recall increasing from 45.5% to 54.2%, and mAP@50 rising from 46.5% to 58.6%. These improvements reflect the combined benefits of dataset expansion and the adoption of newer detection architectures.

Class-wise analysis shows that PET and LDPE consistently achieve the highest detection scores across all models, indicating strong visual distinguishability. In contrast, PP and PVC remain the most challenging categories due to greater inter-class similarity. The normalized confusion matrices reveal that many errors arise from missed detections classified as background, while YOLO26n demonstrates stronger class separation and lower background confusion than RF-DETR.

Overall, while RF-DETR provides the highest absolute detection accuracy, YOLO26n offers the best balance of accuracy, speed, and computational efficiency, making it the preferred model for practical real-time plastic material identification and automated waste segregation systems.

5. CONCLUSION & FUTURE SCOPE

5.1 PROJECT CONCLUSION

The RecycleNet project successfully demonstrates a low-cost, high-speed computer vision pipeline capable of automatically sorting post-consumer plastic waste based on industrial resin identification codes. By utilizing modern edge-focused deep learning structures like YOLO26n, the system delivers real-time inference speeds as low as 2.0 ms. This replaces traditional expensive NIR spectrometer setups with an accessible, multi-platform software ecosystem that runs smoothly on standard desktop environments and offline mobile apps.

One of the current limitations of the system is that YOLO-based object detection models rely heavily on visual shape, contour, and appearance features. As a result, plastics with similar shapes but different material compositions may sometimes be difficult to distinguish accurately. Future research will therefore explore advanced material-aware computer vision models that emphasize texture, surface characteristics, and material property classification rather than shape alone. Integrating such models could significantly improve classification accuracy, robustness, and generalization across a wider range of plastic waste types, further enhancing the effectiveness of automated recycling systems.

5.2 FUTURE SCOPE

Future work will focus on integrating this deep learning software engine directly with physical sorting hardware. By combining the visual output tracking channel with an Arduino or Raspberry Pi microcontroller interface, the classification data can drive mechanical actuator rods or high-pressure pneumatic air nozzle arrays. This extension will create a fully automated, low-cost physical sorting machine capable of physically deflecting classified plastic containers into dedicated material collection bins in real time.

6. APPENDIX

6.1 GETTING STARTED WITH INSTALLATION (ANDROID)

Android Studio (Latest stable version recommended)

Android device or emulator running API 24 or higher (a physical device is highly recommended for CameraX and ML performance).

Installation

Clone the repository:
(git clone https://github.com/yourusername/RecycleNet.git)

Add the PyTorch Model: The application relies on a custom trained YOLO model. You must place your exported PyTorch Lite model into the assets folder before building.

Rename your model to best.ptl.
Place it in the following directory: **app/src/main/assets/best.ptl**. (Note: If the assets folder does not exist, create it).

Build and Run: Sync the Gradle files and run the application on your physical device.

Direct Installation : (**https://github.com/SparshGupta-T/RecycleNet**) **- Get the APK file from this link and install on your device.**

6.2 GETTING STARTED WITH INSTALLATION (DESKTOP)

Requirements
Before you can run the project, ensure you have a Python environment set up with all necessary libraries installed, particularly those required for PyTorch and Ultralytics to utilize macOS Apple Silicon:

pip install flask opencv-python pyyaml ultralytics
pip install torch torchvision torchaudio

Clone the repository:
git clone https://github.com/yourusername/WasteSorting.git

How to Run the Application

1.Open your terminal and navigate to the project directory:
cd ~/Desktop/WasteSorting

2.Start the Flask server:
python3 app.py

3.Open your web browser and navigate to the local server address: **http://127.0.0.1:5000**

Project URL- (**https://github.com/SparshGupta-T/WasteSorting**)

7.REFERENCES

1. He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep Residual Learning for Image Recognition. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), pp. 770-778.
2. Jocher, G., Chaurasia, A., & Qiu, J. (2023). Ultralytics YOLOv8. Available at: https://github.com/ultralytics/ultralytics.
3. Redmon, J., & Farhadi, A. (2018). YOLOv3: An Incremental Improvement. arXiv preprint arXiv:1804.02767.
4. Ren, S., He, K., Girshick, R., & Sun, J. (2015). Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks. Advances in Neural Information Processing Systems (NeurIPS), pp. 91-99.
5. Lin, T. Y., Goyal, P., Girshick, R., He, K., & Dollár, P. (2017). Focal Loss for Dense Object Detection. Proceedings of the IEEE International Conference on Computer Vision (ICCV).
6. [6] S. Majumder, S. K. Gupta, S. Karmakar, B. Roy, M. Bhagat, and H. Saha, "Deep Learning-Assisted Dynamic Plastic Identification Technique," in Proceedings of the 2026 International Conference on Computing, Electronics & Communications Engineering (ICCECE), Kolkata, India, 2026, pp. 1-6, doi: 10.1109/ICCECE69715.2026.11493068.